

Held Objects

Pickup Object

`PickupObject` allows the agent to pick up an interactable object specified by its `objectId`. Compatible objects have `pickable` as `True` in their object metadata and return their current state with `LookupObject`. Note that the agent's hand must be clear of obstruction or the action will fail. If the target object being in the agent's hand would cause it to clip into the environment, the action will also fail.

Picked up objects can also obstruct the Agent's view of the environment since the Agent's hand is always in camera view, so know that picking up larger objects will obstruct the field of vision.

Warning
Certain objects are `receptacles` that can themselves be picked up. If a moveable receptacle is picked up while other Sim Objects are inside of it, the contained objects will be picked up with the moveable receptacle. Here, a sequence like "Place the Egg on the Plate", then to `Pickup` the `Plate` would `Pickup` the `Plate` with Egg on it.

```
controller: step:
  action: "PickupObject",
  objectId: "table011111",
  forceAction: false,
  placeStationary: true
```

Pickup Object Parameters		
objectId	string	required
The target objects <code>objectId</code> , found in the objects' metadata.		
forceAction	bool = False	
Set to <code>True</code> to allow an object to be picked up regardless of if it is within the <code>visibilityDistance</code> of the agent.		
manualInteract	bool = False	
By default, objects picked up by the agent teleport into the agent's hand at a default position in front of the agent camera. Set this to <code>True</code> in order to instead pick up an object at the object's location. This allows the agent to manipulate the object via object manipulation actions without the distraction of the picked up object teleporting to the agent's hand.		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

Put Object

`PutObject` attempts to put an object the agent is holding onto or into the target receptacle. Valid target receptacle objects have `receptacle` as `True` in their object metadata.

```
controller: step:
  action: "PutObject",
  objectId: "table011111",
  forceAction: false,
  placeStationary: true
```

Put Object Parameters		
objectId	string	required
The target objects <code>objectId</code> , found in the objects' metadata.		
forceAction	bool = False	
Enable to ignore any Receptacle Restrictions when attempting to place objects. Normally objects will fail to be put on a receptacle if that receptacle is not valid for the object. This will also ignore interaction range restrictions of the agent. Note this does not guarantee an object will be placed in a receptacle, as some objects will not fit inside a receptacle regardless of the object's object restrictions.		
placeStationary	bool = True	
If <code>False</code> , a placed object will use the physics engine to resolve the final position. This means placing an object on an uneven surface may cause inconsistent results due to the object rolling around or even falling off of the target receptacle. Note that because of variances in physics resolution, this placement mode is non-deterministic. If <code>True</code> , the object will be placed onto the valid receptacle without using physics to resolve the final position. This means that the object will be placed so that it will not roll around. For deterministic placement make sure to set to <code>True</code> .		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

Drop Object

`DropObject` attempts to drop an object currently in the agent's hand and let physics resolve where it lands. The action is different from `PutObject`, as this does not guarantee the held object will be put into a specified receptacle. This is meant to be used in tandem with the `MoveHouse` Hand functions to maneuver a held object to a target area, and the let it drop.

Additionally, this drop action will fail if the held object is not clear from all collisions. Most importantly, the agent's collision will prevent drop, as dropping an object if it is "inside" the agent will lead to unintended behavior.

```
controller: step:
  action: "DropObject",
  forceAction: true
```

Drop Object Parameters		
forceAction	bool = False	
Set to <code>True</code> to forcibly drop an object even if this would cause clipping with the environment, other objects, or the agent.		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

Throw Object

`ThrowObject` is an extension of `DropObject`, it throws the object currently in the agent's hand in the forward facing direction of the agent with a force of `moveMagnitude` newtons. Since objects have different mass properties, objects require different forces to throw them same distance.

```
controller: step:
  action: "ThrowObject",
  moveMagnitude: 8,
  forceAction: true
```

Throw Object Parameters		
moveMagnitude	float	required
The amount of force used to throw the object in newtons. Note that objects of different masses will have different throw distances if this magnitude is not changed.		
forceAction	bool = False	
Set to <code>True</code> to forcibly throw an object even if this would cause clipping with the environment or other objects.		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

Move Held Object

While the agent is holding an object, it has several available actions to manipulate it. One such action is the ability to move the held object closer or further away from it. Held object movement is useful if we want to drop an object on a surface that is relatively far away.

There are several directions that the agent can move the held object. They include: `MoveHeldObjectAhead`, `MoveHeldObjectBack`, `MoveHeldObjectLeft`, `MoveHeldObjectRight`, `MoveHeldObjectUp`, and `MoveHeldObjectDown`. Each direction supports the same additional parameters.

Warning
After calling a move action (e.g., `MoveAhead`) or a rotate action (e.g., `RotateRight`) the position of the held object will reset to its default position in-front of the agent.

```
controller: step:
  action: "MoveHeldObjectAhead",
  moveMagnitude: 1,
  forceVisible: false

# other supported directions
controller: step: "MoveHeldObjectBack"
controller: step: "MoveHeldObjectLeft"
controller: step: "MoveHeldObjectRight"
controller: step: "MoveHeldObjectUp"
controller: step: "MoveHeldObjectDown"
```

Move Held Object Parameters		
moveMagnitude	float	required
The distance, in meters, to move the held object in the specified direction.		
forceVisible	bool = False	
Setting <code>forceVisible=True</code> results in action failing if the object is not visible to the agent. This prevents the agent from hiding the held object inside or behind another object, or moving it too far, such that it's out of the agent's field of view.		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

Rotate Held Object

`RotateHeldObject` attempts to rotate the held object relative to its current rotation.

Warning
After calling a move action (e.g., `MoveAhead`) or a rotate action (e.g., `RotateRight`) the rotation of the held object will reset to its default rotation.

```
controller: step:
  action: "RotateHeldObject",
  pitch: 0,
  yaw: 0,
  roll: 0,
  forceVisible: false
```

Rotate Held Object Parameters		
pitch	float = 0	required
Increments the pitch of an object from the agent's current facing direction. Specified in degrees. The video below shows the held object's <code>pitch</code> , incrementing in 15 degree increments.		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

yaw	float = 0	required
Increments the yaw of an object from the agent's current facing direction. Specified in degrees. The video below shows the held object's <code>yaw</code> , incrementing in 15 degree increments.		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

roll	float = 0	required
Increments the roll of an object from the agent's current facing direction. Specified in degrees. The video below shows the held object's <code>roll</code> , incrementing in 15 degree increments.		

이 동영상상을 시청하려면 Vimeo에 로그인하세요.

만약 주제를 적용해야 할 수도 있습니다. 메시지가 표시되면 해를 선택하여 계속합니다.

로그인

An overload of `RotateHeldObject` can be used to rotate the held object to a fixed rotation.

```
controller: step:
  action: "RotateHeldObject",
  pitch: 0,
  yaw: 0,
  roll: 0,
  forceVisible: false
```

Rotate Held Object Parameters		
rotation: direction: float		required
Sets the rotation of the object to the provided rotation, in degrees. The rotation is relative to the object's axes. So, the set rotation is independent of the object's current rotation. Rotations are specified in degrees.		

Pushing Objects

Directional Push

`DirectionalPush` attempts to push an object in a given direction. Since objects can have different mass properties, pushing different objects often requires differing amounts of force to move them the same distance. Only `pickable` and `moveable` objects can be pushed.

```
controller: step:
  action: "DirectionalPush",
  objectId: "table011111",
  moveMagnitude: 10,
  pushAngle: 30
```

Directional Push Parameters		
moveMagnitude	float	required
The amount of force used to push the object in newtons. Following natural physics, objects of different masses may move different distances if this magnitude is not changed.		
pushAngle	float	required
The direction vector to push the object. Values in [0 : 360] are valid, with 0 being the current forward direction of the agent. This value will change the push direction clockwise from the agent's forward, (i.e., 90 will push the object directly right, a value of 180 will push the object backwards).		
objectId	str	required
The target object's <code>objectId</code> , found in the objects' metadata.		
forceAction	bool = False	
Set to <code>True</code> to allow an object to be picked up regardless of if it is within the <code>visibilityDistance</code> of the agent.		

Push Object

`PushObject` is syntactic sugar for `DirectionalPush` with a `pushAngle` of 0.

```
controller: step:
  action: "PushObject",
  objectId: "table011111",
  forceAction: false
```

Push Object Parameters		
moveMagnitude	float	required
The amount of force used to push the object in newtons. Note that objects of different masses will move different distances if this magnitude is not changed.		
objectId	str	required
The target object's <code>objectId</code> , found in the objects' metadata.		
forceAction	bool = False	
Set to <code>True</code> to allow an object to be picked up regardless of if it is within the <code>visibilityDistance</code> of the agent.		

Pull Object

`PullObject` is syntactic sugar for `DirectionalPush` with a `pushAngle` of 180.

```
controller: step:
  action: "PullObject",
  objectId: "table011111",
  forceAction: false
```

Pull Object Parameters		
moveMagnitude	float	required
The amount of force used to pull the object in newtons. Note that objects of different masses will move different distances if this magnitude is not changed.		
objectId	str	required
The target object's <code>objectId</code> , found in the objects' metadata.		
forceAction	bool = False	
Set to <code>True</code> to allow an object to be picked up regardless of if it is within the <code>visibilityDistance</code> of the agent.		

Touch Then Apply Force

`TouchThenApplyForce` allows the agent to push an object at a certain point on the object, in a given direction, if a `pickable` or `moveable` sim object is hit along the path of this ray. It will have a force determined by `moveMagnitude` applied to it instantaneously. This action will return feedback in the `actionReturn` attribute of the event metadata return.

```
event: controller: step:
  action: "TouchThenApplyForce",
  x=0.5,
  y=0.5,
  direction: {
    "x": 0.0,
    "y": 0.0,
    "z": 0.0
  },
  moveMagnitude: 1,
  handDistance: 1.5
```

Parameters		
x	float	required
Used instead of <code>objectId</code> as an alternate way to target an object based on the last image frame. Valid values are in [0 : 1], corresponding to how far left the image is from the previous frame.		
y	float	required
Used in tandem with 'x' to target an object based on the last image frame, rather than an <code>objectId</code> . Valid values are in [0 : 1], corresponding to how far left the image is from the previous frame.		
direction	dict(str, float)	required
The direction vector relative to the agent's current forward to push any object touched.		
moveMagnitude	float	required
The amount of force to apply to a touched object in newtons.		
handDistance	float	required
The maximum Euclidean distance from the agent's camera that the (x, y) point that targets the object can be, in meters. If the point on the object is further away than <code>handDistance</code> , the action will fail.		

```
Response
{
  "didHandTouchSomething": True
  "objectId": "apple011111"
  "armLength": 1.28
}
```

event.metadata["actionReturn"]		
didHandTouchSomething	bool	
Returns <code>True</code> if a sim object was touched within the ray's length of <code>handDistance</code> meters.		
objectId	str	
The unique string of the object touched by the ray. If the object touched in the scene has collision but is not a sim object, this <code>objectId</code> attribute will be "not a sim object, a structure was touched".		
armLength	float	
The distance from the touched point on the object to the agent's camera. This distance will never exceed the <code>handDistance</code> value passed in originally unless the action finishes as a failed action (see below).		

This action will return a failure `[event.metadata["actionReturn"] == False]` only in two cases:

- The raycast hit an object, but the object was outside of the maximum visibility range of the agent (ie: a `handDistance` of 10 is passed in, and hits an object 9 meters away, but the agent's max visibility distance is 1.5 meters, causing a failure). The feedback object generated in this case will be `didHandTouchSomething=False`, `objectId=""`, `armLength=handDistance`.
- The `handDistance` of the action is larger than the agent's max visibility distance and no object was hit. The feedback object generated in this case will be `didHandTouchSomething=False`, `objectId=""`, `armLength=handDistance` with a metadata error message of "the position the hand would have moved to is outside the agent's max interaction range".

Note that this action interacts with the visibility of an object in order to determine what can be picked. The visibility of an object is defined by:

- An object must be within the agent's camera's field of view.
- An object must be within the area of a cylinder defined by a radius of length `visibilityDistance` around the agent's vertical axis.
- If that object is within the cylinder, a line must be able to be cast from the agent's camera position to a point on that object unobstructed.

Because visibility is defined by the cylinder with radius `visibilityDistance`, the total area of objects that are touchable by this action is the intersection of the sphere of radius `handDistance` centered around the agent's camera, and the cylinder of radius `visibilityDistance` around the agent's vertical axis. Note that the agent camera is also centered around the agent's vertical axis.

Teleporting Objects

Place Object At Point

`PlaceObjectAtPoint` attempts to place an object flush with the surface of a receptacle. This can only be used on objects that are `pickable` or `moveable`. The point to place an object at can be generated by the `GetSpawnCoordinatesAboveReceptacle` action below. Combine these two actions in order to teleport objects onto different receptacle surfaces.

```
controller: step:
  action: "PlaceObjectAtPoint",
  objectId: "table011111",
  position: {
    "x": -1.25,
    "y": 1.0,
    "z": -1.0
  }
}
```

Place Object At Point Parameters		
objectId	str	required
The target object's <code>objectId</code> , found in the objects' metadata.		
position	dict(str, float)	required
Global (x, y, z) coordinates for the point to try and place the object.		

Get Spawn Coordinate Above Receptacle

`GetSpawnCoordinatesAboveReceptacle` is to explicitly be used in tandem with the `PlaceObjectAtPoint` action. It returns an array of xyz dictionary representing the (x, y, z) coordinates of valid spawn positions above a `receptacle` object. The array of xyz dictionary is returned in the `actionReturn` metadata.

```
controller: step:
  action: "GetSpawnCoordinatesAboveReceptacle",
  objectId: "table011111",
  anywhere: false
```

Parameters		
objectId	str	required
The target object's <code>objectId</code> , found in the objects' metadata.		
anywhere	bool = False	
If <code>True</code> , spawn coordinates will be returned even if the exact position of the coordinate is outside of the agent's field of view. Keep <code>False</code> to return spawn coordinates even if they are not in view of the agent.		